

Visão geral honesta

A implementação atual é **majoritariamente uma orquestração manual por prompts + Edge Functions + Postgres**.

Há uma **arquitetura conceitual** de agentes no README e nos prompts, mas o fluxo real faz, na maior parte, **uma chamada LLM por etapa**.

1) Na solução, é utilizado algum framework para gestão dos agentes? Qual?

Não encontrei framework dedicado de agent orchestration como LangChain, LangGraph, CrewAI, AutoGen, Semantic Kernel ou LlamaIndex.

O que existe hoje é:

- **Edge Functions em Deno** para orquestrar as chamadas
- **Prompts grandes e estruturados** simulando papéis/agentes
- **Supabase/Postgres** como memória operacional e persistência
- **Function calling** para forçar saída estruturada em JSON

Evidências no projeto:

- `package.json` não traz libs de orquestração de agentes
- o fluxo principal da análise chama **uma única edge function**:
 - `src/pages/app/NewAnalysisPage.tsx:577-602`
- essa edge function faz **uma única chamada LLM** com tool/function calling:
 - `supabase/functions/analyze-campaign/index.ts:279-496`
- existe catálogo de agentes no banco:
 - `supabase/migrations/20260315110350_e80098f3-a110-4d25-8201-2ff226b86002.sql:18-23`
 - `...:203-243`
- mas eu **não encontrei no repositório uma execução real separada por agente** nem inserts consistentes em `agent_responses`; o que há é mais uma modelagem preparada para isso do que um runtime completo já implementado.

Resumo técnico:

o “multi-agente” do Ágora hoje é **prompt-driven**, não **framework-driven**.

2) Como é feita a engenharia de contexto?

A engenharia de contexto é feita por **montagem manual de prompt** e por **rehidratação de contexto a partir do Postgres**.

Como isso acontece na prática

a) Intake conversacional

O chat inicial envia:

- histórico de mensagens
- prompt de sistema enorme com a arquitetura conceitual dos agentes
- arquivos do usuário como:
 - **texto bruto**, quando o arquivo é texto
 - **base64 inline**, quando não é texto

Arquivos relevantes:

- leitura local dos arquivos no front:
 - `src/pages/app/NewAnalysisPage.tsx:287-305`
- envio do histórico + arquivos para a edge function:
 - `src/pages/app/NewAnalysisPage.tsx:40-58`
- injeção dos arquivos no prompt:
 - `supabase/functions/intake-chat/index.ts:555-603`

Ou seja: aqui o contexto é montado por **prompt stuffing direto**.

b) Enriquecimento por dados externos

Na análise e otimização, o backend tenta extrair região do texto e buscar dados no **IBGE/SIDRA** para enriquecer o contexto.

- `supabase/functions/analyze-campaign/index.ts:21-173`
- `supabase/functions/analyze-campaign/index.ts:231-277`
- `supabase/functions/optimize-campaign/index.ts:43-76`
- `supabase/functions/optimize-campaign/index.ts:182-226`

c) Persistência do contexto canônico

Depois da análise, o sistema salva no Postgres:

- `raw_prompt`
- `scores`
- `industry, region, primary_channel, declared_target_audience`
- `normalized_payload` com resumo executivo, melhorias, forças, vieses, etc.
- `schema`:
 - `supabase/migrations/...10350...sql:157-180`
- `gravação`:
 - `src/pages/app/NewAnalysisPage.tsx:605-632`
 - `src/pages/app/CampaignOptimizerPage.tsx:242-256`

d) Rehidratação do contexto em etapas posteriores

Depois, outras features usam o Postgres como memória operacional:

- **Chat com estrategista** monta contexto a partir da análise salva:
 - `src/pages/app/AnalysisChatPage.tsx:167-183`
 - `supabase/functions/strategist-chat/index.ts:41-77`
- **Geração de criativo** busca:
 - `analysis_requests`
 - `chat_messages`
 - `agent_responses` (se existirem)
 e transforma isso num `creativeContext`:
 - `supabase/functions/generate-creative/index.ts:45-103`
- **Edição de documento** injeta o documento inteiro atual no prompt:
 - `supabase/functions/campaign-chat/index.ts:30-37`

e) Controle manual de tamanho

Há alguns cortes manuais, por exemplo:

- chat history limitado aos últimos 20 itens e truncado:
 - `supabase/functions/generate-creative/index.ts:58-81`

Resumo técnico:

a engenharia de contexto é feita por:

1. **prompt assembly manual**
2. **persistência em Postgres**
3. **rehidratação seletiva**
4. **truncagem manual**

Não vi mecanismo mais sofisticado de context compression, reranking, memory selection semântica ou graph memory.

3) Qual a função do Postgres?

O Postgres é o **backbone operacional** da aplicação.

Ele serve para:

a) Persistir o estado do produto

- usuários/perfis/planos
- empresas e membros
- limites de uso
- integrações

b) Persistir o estado analítico

- campanhas analisadas em `analysis_requests`
- payload estruturado em JSONB
- outputs gerados
- conversas
- criativos gerados

Tabelas principais:

- `analysis_requests` :
 - `...10350...sql:157-180`
- `files` :
 - `...10350...sql:185-200`
- `agents` :
 - `...10350...sql:203-225`
- `agent_responses` :
 - `...10350...sql:227-243`
- `generated_outputs` :
 - `...10350...sql:245-257`
- `conversations / chat_messages` :
 - `supabase/migrations/20260315144106_031f02f9-49b5-4420-8d21-589cae265c94.sql:2-18`

- `creative_jobs :`
 - `supabase/migrations/20260315165804_b0920c2b-8df8-42b6-b06e-1ff92dale20b.sql:2-15`

c) Servir como memória de contexto

O Postgres guarda o material que depois é reusado nos prompts:

- `normalized_payload`
- `histórico de conversa`
- `prompt_context`
- `strategist_output`

d) Segurança

O banco usa **RLS** para isolar os dados por usuário:

- `...10350...sql:337-400`

e) Semi-estruturação com JSONB

O projeto usa bastante JSONB e até GIN index para isso:

- `...10350...sql:334-335`

Resumo técnico:

o Postgres aqui é **banco transacional + memória operacional + store de contexto estruturado**, e não um vector DB.

4) É realizado RAG? Se sim, é vetorial? Há chunking/chunk overlap para documentos não estruturados?

Resposta curta

Não há um RAG clássico implementado no código atual.

O que existe de fato

Existe:

- **injeção direta de contexto no prompt**
- **rehidratação de dados já salvos no Postgres**

- **envio inline de arquivo** no intake

Mas eu **não encontrei**:

- embeddings
- pgvector
- colunas vetoriais
- busca por similaridade
- retriever
- reranker
- pipeline de chunking
- chunk overlap

Sinais concretos disso

Não há vetor

No schema, a extensão criada é `pgcrypto`, não `vector`:

- `...10350...sql:2-4`

Também não encontrei consultas de similaridade ou embeddings no repositório.

Não há chunking

Arquivos no intake são tratados assim:

- texto: vai inteiro como texto
- não texto: vai como base64 inline
- `src/pages/app/NewAnalysisPage.tsx:287-305`
- `supabase/functions/intake-chat/index.ts:563-603`

Isso não é chunking; é **injeção bruta**.

A análise final nem usa o conteúdo do arquivo

No fluxo principal de análise, o front envia **apenas os nomes dos arquivos**:

- `src/pages/app/NewAnalysisPage.tsx:575-589`

E o backend usa isso só como lista textual:

- `supabase/functions/analyze-campaign/index.ts:267-275`

Mesma lógica no `optimize-campaign`:

- `src/pages/app/CampaignOptimizerPage.tsx:223-226`
- `supabase/functions/optimize-campaign/index.ts:206-226`

Então, no estado atual, o conteúdo documental **não entra de forma robusta** na análise principal.

Há campo para texto extraído, mas não vi pipeline ativo

Existe `files.extracted_text` no schema:

- `...10350...sql:197`

Mas eu **não encontrei no repositório uma rotina que preencha ou consulte esse campo.**

Conclusão sobre RAG

Tecnicamente, eu responderia assim:

- **Não há RAG vetorial implementado**
 - **Não há chunking/chunk overlap**
 - O que existe hoje é mais próximo de:
 - **prompt stuffing**
 - **context rehydration**
 - **structured context reuse**
-

5) Se possível, apresentar um desenho da arquitetura dos agentes/chamadas LLMs

Arquitetura conceitual vendida pelo projeto

O README e o prompt de intake descrevem algo assim:

- Orquestrador Master
- Agente Sociocomportamental
- Engenheiro de Oferta
- Cientista de Performance
- Estrategista-Chefe

Referências:

- `README.md:68-112`

- `supabase/functions/intake-chat/index.ts:12-139`

Arquitetura implementada hoje

O desenho real do código está mais próximo disto:

```
[Browser / React]
|
| 1) Chat de intake + arquivos
v
[intake-chat Edge Function]
- monta prompt gigante
- injeta histórico + texto/base64 de arquivos
- chama Gemini 2.5 Flash
- devolve streaming + marcador ##READY##
|
v
[Postgres]
- conversations
- chat_messages
```

Quando usuário inicia análise:

```
|
v
[analysis_requests + files]
- salva raw_prompt
- salva metadata de arquivos no storage
|
v
[analyze-campaign Edge Function]
- tenta enriquecer com IBGE/SIDRA
- monta prompt de análise
- chama Gemini 2.5 Flash com function calling
- retorna JSON estruturado
|
v
[Postgres]
- analysis_requests.normalized_payload
- scores
- industry / region / channel / audience
```

A partir daí:

```
|
+--> [strategist-chat]
|       - reidrata analysisContext
|       - chama Gemini 2.5 Flash
```

```
|
+--> [generate-campaign]
|     - usa analysisData + improvements
|     - chama Gemini 2.5 Flash (streaming markdown)
|
+--> [campaign-chat]
|     - injeta currentDocument inteiro
|     - chama Gemini 2.5 Flash
|
+--> [generate-creative]
|     - lê analysis_requests + chat_messages (+ agent_responses, se
houver)
|     - monta creativeContext
|     - chama Gemini 3 Flash Preview para briefing
|     - chama Gemini 2.5 Flash Image para imagem
|     - salva em creative_jobs
```

Conclusão final

Se eu fosse responder de forma técnica e objetiva para alguém avaliando o Ágora, eu diria:

O Ágora atual não usa um framework formal de gestão de agentes. A “arquitetura multi-agente” é implementada principalmente por prompts estruturados e edge functions serverless sobre Supabase/Postgres. O Postgres atua como memória operacional e store de contexto estruturado. Não identifiquei RAG vetorial, embeddings, pgvector, chunking ou overlap. O sistema hoje funciona mais como uma orquestração manual de chamadas LLM com rehidratação de contexto do banco do que como uma stack clássica de agentes autônomos com retrieval semântico.

Observação importante: o `README.md` menciona **Gemini 2.5 Pro** e **GPT-5**, mas nas edge functions que li o que aparece em runtime é principalmente **Gemini 2.5 Flash**, **Gemini 2.5 Flash Lite**, **Gemini 2.5 Flash Image** e **Gemini 3 Flash Preview**.